codecademy

Introduction to Django

What is Django?

Django is a web framework written in the Python programming language aimed at building web applications quickly. Django is free, opened source, and maintained by Django Software Foundation.

Opinionated Frameworks

Opinionated frameworks recommend specific ways of solving problems to streaming productivity.

Starting a Development Server

A project installation comes with administrative tools to deploy a development server immediately after establishing a project.

```
python3 manage.py runserver
```

View Functions

URLs are mapped to a view function, which processes a user's web request and sends a response back.

```
def home(request):
  return HttpResponse("This is a response from the server.")
```

Projects and Apps

A project is a collection of configurations and individual apps for a particular website. Each app should have a clearly defined purpose.

django-admin Utility

A project is started using the `startproject` command.

```
django-admin startproject myproject
```

manage.py

manage.py is a file that includes commands to administer a Django app.

settings.py

The **settings.py** file contains the project settings for a Django project.

Starting a Development Server

A project installation comes with administrative tools to deploy a development server immediately after establishing a project.

```
python3 manage.py runserver
```

Stopping a Django Server

A Django server can be stopped by pressing `ctrl` + `c`.

Development Server Port

The development server runs locally on port 8000 by default. An alternative port can be provided as an option.

```
python3 manage.py runserver
<optional_port_number>
```

Viewing the Project

When the development server is started on the default port, the project can be viewed in the browser at the URL: `http://localhost:8000`.

Hot Reload

The development server will reload whenever there are changes to the application code.

Default Apps

New Django projects have several apps installed by default such as the admin panel and authentication.

```
INSTALLED_APPS = [
# ... Other pre-installed apps
  "myapp.apps.MyappConfig"
]
```

Creating a Django App

New apps are made with the `startapp` command.

```
python3 manage.py startapp <name_of_app>
```

Installed Apps

New Django apps must be added to **settings.py** in the `INSTALLED_APPS` list in order for the project to recognize them.

```
INSTALLED_APPS = [
# ... Other pre-installed apps
  "myapp.apps.MyappConfig"
]
```

URLconfig Files

URLs are configured in URLconfig file, **urls.py**.

```
urlpatterns = [
  ("", include("app.urls"))
]
```

Mapping View Functions

View functions are mapped to URLs inside **urls.py**.

```
urlpatterns = [
  path('', views.home, name="home")
]
```

Migrations

The `migrate` command will update database models for projects installed in `INSTALLED_APPS`.

```
python3 manage.py migrate
```

urlpatterns

URLs are created using the `path()` function which takes three arguments— the route, the view function, and an optional name.

```
path("", views.home, name="home")
```

Template Namespace

Templates should be namespaced by creating a nested folder structure to avoid name conflicts between apps.

HTTP Responses

A response to a web request is given in a `HttpResponse` object, which can be anything from HTML to an image.

```
def myView(request):
  return HttpResponse("Here is my
response")
```

HTML Front-End

The front-end refers to the part of web apps visible to users, handled in their browsers. This includes elements like HTML for structure, CSS for styling, and JavaScript for functionality. Meanwhile, the back-end handles the server-side processes, managing data and responding to requests.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Front-End Development</title>
   <style>
       body {
           font-family: Arial, sans-serif;
           background-color: #f4f4f9;
       }
   </style>
</head>
<body>
   <h1>Hello, World!</h1>
   <p>This is a simple HTML page
demonstrating front-end basics.</p>
   <button onclick="alert('Hello, Front-
End!')">Click me!</button>
</body>
</html>
```

Web Server Basics

A **web server** listens for client requests over a network and returns responses. Typically, these servers use the **HTTP protocol** to send requested resources like HTML pages, images, or files. Understanding these basic functions of a web server helps in designing efficient network communication.

```python
# Simple HTTP server in Python
import http.server
import socketserver

PORT = 8000

Handler =
http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer(("", PORT),
Handler) as httpd:
    print(f"Serving at port {PORT}")
    httpd.serve_forever()
```

Server-Side Logic

The back-end of web applications handles server-side logic, processing requests and managing data interactions. This ensures dynamic responses are generated for clients effectively. `Node.js` is often used for this purpose, working seamlessly with databases like MongoDB or MySQL.

```javascript
const http = require('http');
const server = http.createServer((req,
res) => {
  if (req.url === '/') {
    res.write('You have reached the
server-side logic page');
    res.end();
  }
});
server.listen(3000);
console.log('Listening on port 3000...');
```

`Database` Management

Databases store and manage data for web apps efficiently. Relational databases organize data into tables, facilitating easy access and manipulation using queries. Non-relational databases store data differently, like key-value pairs or documents, offering flexibility for unstructured data.

```
--Example of a relational database query:
SELECT * FROM users WHERE status =
'active';

--Example of non-relational database entry
using JSON-like format:
{
   "key": "user123",
   "value": {
      "name": "John Doe",
      "status": "active"
   }
}
```

Understanding Web APIs

A **web API** enables communication between a client interface and a server by following a specific set of rules. Through a process called the HTTP request-response cycle, clients can retrieve, update, or manipulate data on the server side. This interaction is essential for dynamic web applications that rely on external data.

```
GET /api/data HTTP/1.1
Host: example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
{
    "data": [
        {"id": 1, "name": "John Doe"},
        {"id": 2, "name": "Jane Smith"}
    ]
}
```

`Python` Authentication

Authentication ensures only legitimate users can access specific features in an application by verifying their identity, often using credentials, like usernames and passwords.

```
from getpass import getpass

# Placeholder user credentials for
demonstration
USERNAME = "user123"
PASSWORD = "securepass"

username_input = input("Enter your
username: ")
password_input = getpass("Enter your
password: ")

if username_input == USERNAME and
password_input == PASSWORD:
    print("Access granted.")
else:
    print("Access denied.")
```

`Authorization` Concepts

Authorization is critical for controlling access. Once a user proves their identity, authorization determines what they can do. It's like having different keys for different doors based on your role!

```
// Example of authorization check
const user = {
  role: 'admin',
  permissions: ['view-dashboard', 'edit-user'],
};

function authorize(action) {
  if (user.permissions.includes(action)) {
    return `Access granted for ${action}`;
  } else {
    return `Access denied for ${action}`;
  }
}

console.log(authorize('edit-user'));  // Outputs: Access granted for edit-user
console.log(authorize('delete-user')); // Outputs: Access denied for delete-user
```

`Tech Stack` Overview

A **technology stack** is an essential part of web development. It combines **programming languages**, **frameworks**, **databases**, and **tools** to create a cohesive environment for building web applications. Understanding how each component interacts can enhance productivity and efficiency.